



INGENIERÍA EN DESARROLLO Y GESTIÓN DE SOFTWARE

EXTRACCIÓN DEL CONOCIMIENTO EN BASES DE DATOS

T2. KNN y SVM

PRESENTA:

BERTADILLO VILLALOBOS LEOBARDO DANIEL

GRUPO: 9° C

Profesora:

VIRNA VIRIDIANA VELA RINCÓN

Emiliano Zapata, Morelos, México.

Resumen

En este trabajo se prueba si una computadora puede reconocer el tipo de vehículo a partir de la forma de su silueta. Para ello se usó el conjunto de datos *Statlog (Vehicle Silhouettes)*, que tiene **846 ejemplos**, cada uno descrito con **18 medidas** de su silueta y agrupado en **4 tipos** de vehículo en cantidades parejas. Se compararon dos algoritmos muy conocidos: **K-Vecinos más Cercanos (KNN)** y **Máquinas de Vectores de Soporte (SVM)**. Después de poner todas las medidas en una escala comparable y separar los datos para entrenar y para probar, se eligió el mejor valor de k para KNN y la mejor configuración para SVM. El mejor KNN ($k = 7$) acertó el **70.28 %** de los casos (con un *Macro F1* de **0.6985**), mientras que SVM obtuvo el mejor resultado con un **86.32 %** de aciertos (*Macro F1* de **0.8646**). Al revisar dónde se equivocan, se observa que casi todos los errores ocurren entre los dos tipos de automóvil, cuyas siluetas son muy parecidas.

Palabras clave: clasificación multiclase, KNN, SVM, kernel RBF, estandarización, validación cruzada, matriz de confusión.

Índice

1. Introducción y Marco Teórico	1
1.1. Contexto del problema	1
1.2. K-Vecinos más Cercanos (KNN)	1
1.3. Máquinas de Vectores de Soporte (SVM)	2
2. Metodología y Desarrollo	2
2.1. Descripción del conjunto de datos	2
2.2. Diseño experimental	3
3. Resultados y Discusión	5
3.1. Comparación de desempeño	5
3.2. Análisis de las matrices de confusión	5
3.3. Cálculo manual de las métricas	6
3.3.1. Desarrollo para el modelo SVM	7
3.3.2. Desarrollo para el modelo KNN	7
4. Conclusiones	8

1. Introducción y Marco Teórico

1.1. Contexto del problema

Enseñar a una computadora a reconocer la forma de un objeto a partir de números que describen su figura es una tarea muy común en el análisis de datos. En este trabajo se resuelve un problema de **clasificación con varias categorías** usando el conjunto de datos *Statlog (Vehicle Silhouettes)*. En él, cada vehículo se describe con 18 medidas tomadas de su silueta (qué tan compacta es, qué tan redonda, su tamaño, etc.), y la meta es adivinar a cuál de los cuatro tipos de vehículo pertenece usando solo esas medidas.

Lo interesante de este conjunto es que dos de las cuatro clases son automóviles de pasajeros con siluetas casi idénticas, mientras que las otras dos son vehículos más grandes y con formas más fáciles de distinguir. Por eso resulta un buen caso para poner a prueba qué tan bien logran separar los datos distintos algoritmos cuando las clases se parecen mucho entre sí.

1.2. K-Vecinos más Cercanos (KNN)

La idea de KNN es muy sencilla: no aprende un modelo durante el entrenamiento, simplemente guarda todos los datos. Cuando llega un caso nuevo $\mathbf{x}$ que quiere clasificar, mide qué tan lejos está de todos los ejemplos guardados, se queda con los k más cercanos (sus “vecinos”) y le asigna la clase que más se repite entre ellos; es decir, decide por **votación mayoritaria**.

La forma más común de medir esa cercanía es la **distancia euclidiana**, que no es otra cosa que la distancia “en línea recta” entre dos puntos. Para dos observaciones $\mathbf{x} = (x_1, \dots, x_p)$ y $\mathbf{x}' = (x'_1, \dots, x'_p)$ con p características se calcula así:

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{j=1}^p (x_j - x'_j)^2}. \quad (1)$$

La clase final $\hat{y}$ es simplemente la más frecuente entre los k vecinos más cercanos $N_k(\mathbf{x})$:

$$\hat{y} = \arg \max_c \sum_{i \in N_k(\mathbf{x})} \mathbb{I}(y_i = c), \quad (2)$$

donde $\mathbb{I}(\cdot)$ es la función indicadora que vale 1 cuando la condición se cumple y 0 en caso contrario.

Impacto de la elección de k

El valor de k cambia mucho el resultado. Si k es muy pequeño (por ejemplo $k = 1$), el modelo se fija solo en el vecino más cercano y se vuelve muy sensible al ruido, equivocándose ante datos atípicos. Si k es demasiado grande, toma en cuenta vecinos lejanos que poco tienen que ver con el caso y pierde detalle. Por eso conviene probar varios valores y quedarse con el que mejor funcione.

Como esta distancia depende del tamaño de los números, KNN es muy sensible a la escala de las variables: si una medida toma valores mucho más grandes que las demás, terminaría mandando en el cálculo. Por eso es importante **estandarizar** las variables (ponerlas en una escala comparable) antes de usar el algoritmo.

1.3. Máquinas de Vectores de Soporte (SVM)

SVM busca trazar la mejor línea (o, cuando hay más variables, el mejor **plano de separación**) que divida a las clases. De todas las separaciones posibles, elige la que deja **el mayor espacio libre** a ambos lados, es decir, la que queda lo más lejos posible de los ejemplos de cada clase. Los pocos ejemplos que caen justo en el borde y que definen esa frontera se llaman **vectores de soporte**; los demás no afectan el resultado.

Cuando las clases se pueden separar con una línea recta, esa frontera se escribe como $\mathbf{w}^\top \mathbf{x} + b = 0$, y entrenar el modelo equivale a resolver:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{sujeto a} \quad y_i (\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n. \quad (3)$$

En la práctica los datos casi nunca se separan de forma perfecta, así que se añade un parámetro C que deja al modelo tolerar algunos errores con tal de lograr una separación más amplia y general. Un C grande castiga más los errores; uno pequeño es más permisivo.

El truco del kernel (*Kernel Trick*)

Muchas veces las clases no se pueden separar con una simple línea recta. En esos casos, SVM “lleva” los datos a un espacio con más dimensiones, donde sí es posible separarlos. Lo ingenioso es que no necesita hacer esa transformación de forma directa (sería muy costosa): usa una **función kernel** que calcula el resultado como si los datos ya estuvieran en ese nuevo espacio. Una de las más usadas es el kernel *Radial Basis Function* (RBF):

$$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2), \quad \gamma > 0. \quad (4)$$

El parámetro γ controla qué tanto influye cada ejemplo: valores altos hacen fronteras más ajustadas a los datos y valores bajos, fronteras más suaves.

2. Metodología y Desarrollo

2.1. Descripción del conjunto de datos

El archivo `vehicle.csv` tiene **846 filas** y **19 columnas**: 18 medidas numéricas de la silueta del vehículo y una columna **target** con el tipo de vehículo al que pertenece. La Tabla 1 muestra cuántos ejemplos hay de cada clase; como están bastante parejas, la exactitud (el porcentaje de aciertos) es una buena forma de medir el desempeño. Además, el conjunto no tiene datos faltantes.

Tabla 1: Distribución de instancias por clase en `vehicle.csv`.

Clase	Instancias	Proporción
1	212	25.06 %
2	217	25.65 %
3	218	25.77 %
4	199	23.52 %
Total	846	100 %

La Figura 1 confirma visualmente este balance entre las cuatro clases.

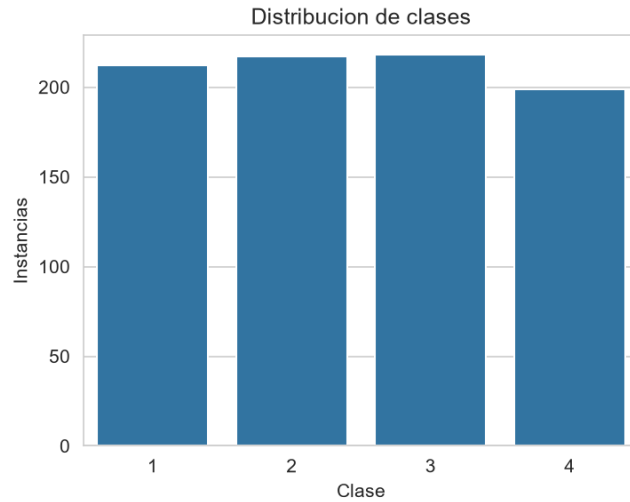


Figura 1: Distribución de instancias por clase.

Importancia de la estandarización

Las 18 medidas usan rangos muy distintos: algunas llegan a valores de cientos y otras apenas a decenas. Como KNN y SVM se basan en distancias, las variables con números más grandes pesarían más que las demás sin ninguna razón. Para evitarlo se aplicó **estandarización**: a cada variable se le resta su promedio y se divide entre su desviación, dejándolas a todas en una escala comparable para que aporten por igual.

2.2. Diseño experimental

El experimento se implementó en Python con la biblioteca `scikit-learn`. El procedimiento fue el siguiente:

1. **División de los datos.** Se separó el conjunto en **75 % para entrenar** (634 ejemplos) y **25 % para probar** (212 ejemplos), cuidando que ambas partes mantuvieran la misma proporción de clases (partición **estratificada**). Se fijó una semilla aleatoria para que el experimento dé siempre el mismo resultado al repetirlo.

2. **Estandarización.** La escala se calculó **solo con los datos de entrenamiento** y luego se aplicó a los de prueba, para que el modelo no reciba de antemano ninguna información de los datos con los que será evaluado.
3. **Selección de k en KNN.** Se evaluaron valores de k de 1 a 30, estimando para cada uno la exactitud mediante **validación cruzada de 5 particiones** sobre el conjunto de entrenamiento. Se eligió el k con mayor exactitud media (Figura 2).
4. **Selección del kernel en SVM.** Se realizó una **búsqueda en rejilla** (*grid search*) con validación cruzada de 5 particiones sobre los kernels lineal, polinomial y RBF, explorando distintos valores de C , γ y grado del polinomio.
5. **Evaluación final.** Ya con su mejor configuración, ambos modelos se probaron con los datos de prueba —que no habían visto antes— midiendo exactitud, *Macro* F1, resultados por clase, matriz de confusión y tiempos de entrenamiento y predicción.

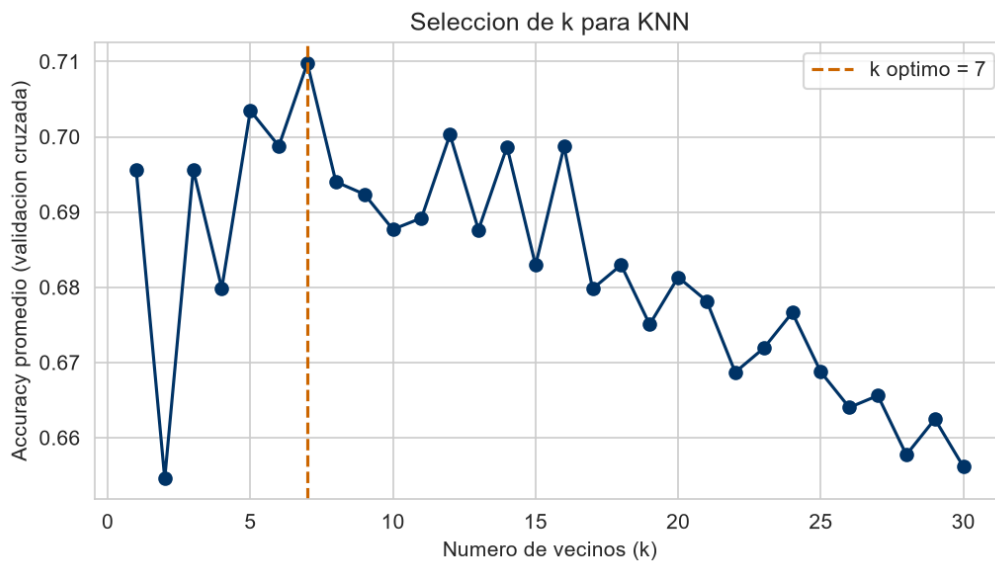


Figura 2: Exactitud media por validación cruzada en función de k . El máximo se alcanza en $k = 7$.

La validación cruzada determinó $k = 7$ como mejor configuración para KNN. La búsqueda en rejilla seleccionó para SVM un **kernel RBF** con $C = 100$ y $\gamma = 0.01$, que superó claramente a las alternativas lineal y polinomial.

3. Resultados y Discusión

3.1. Comparación de desempeño

La Tabla 2 resume el rendimiento de ambos clasificadores sobre el conjunto de prueba. SVM supera a KNN en más de 16 puntos porcentuales de exactitud y en una diferencia equivalente de *Macro* F1, lo que indica un mejor desempeño no solo global sino también de manera equilibrada entre las cuatro clases.

Tabla 2: Comparación de KNN y SVM sobre el conjunto de prueba (212 instancias).

Modelo	Accuracy	Macro F1	T. entren. (ms)	T. predic. (ms)
KNN ($k = 7$)	0.7028	0.6985	2.66	33.19
SVM (RBF)	0.8632	0.8646	20.19	8.07

Los tiempos muestran cómo trabaja cada método. KNN casi no tarda en “entrenar” (solo guarda los datos), pero es el más lento al predecir, porque tiene que comparar cada caso nuevo contra todos los ejemplos guardados. SVM tarda más en entrenar, pero después predice más rápido, ya que solo usa los vectores de soporte. Como el conjunto es pequeño, los tiempos son de milisegundos y conviene verlos como una comparación **relativa**, no como valores absolutos.

La Tabla 3 desglosa el F1 por clase, evidenciando que ambos modelos clasifican casi a la perfección las clases 3 y 4, mientras que la mayor dificultad se concentra en las clases 1 y 2.

Tabla 3: F1-Score por clase para cada modelo.

Modelo	Clase 1	Clase 2	Clase 3	Clase 4
KNN ($k = 7$)	0.4646	0.5586	0.9138	0.8571
SVM (RBF)	0.7568	0.7500	0.9818	0.9697

3.2. Análisis de las matrices de confusión

Las Figuras 3(a) y 3(b) presentan las matrices de confusión de cada modelo. En ambas, la diagonal principal (aciertos) es dominante para las clases 3 y 4, pero el bloque correspondiente a las clases 1 y 2 concentra la mayoría de los errores.

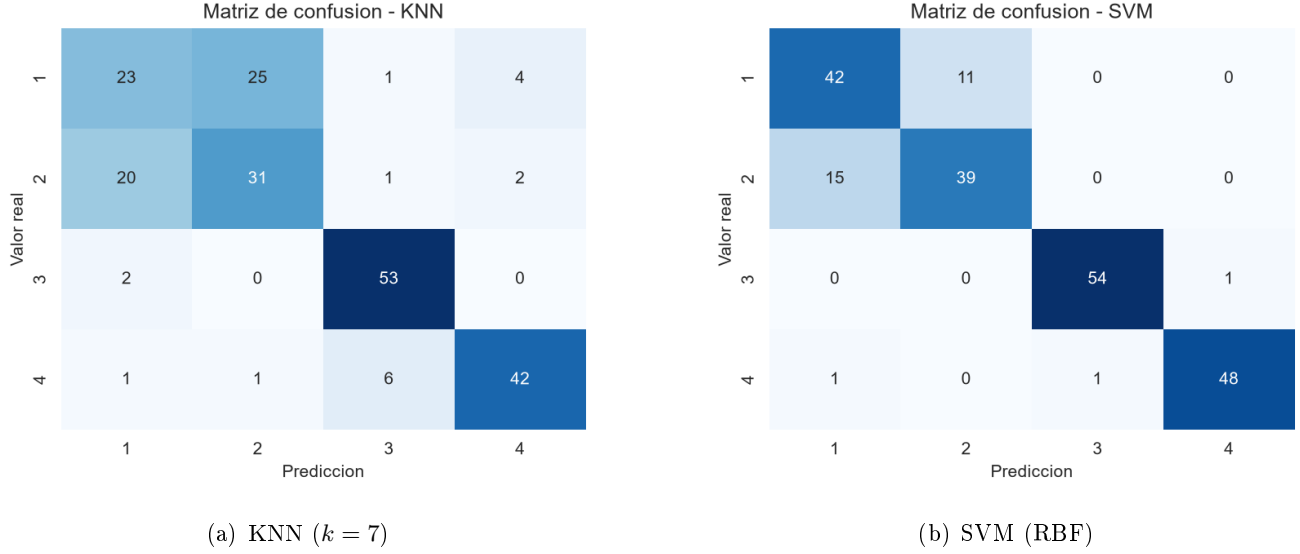


Figura 3: Matrices de confusión sobre el conjunto de prueba.

En el caso de KNN, de las 53 instancias de la clase 1, **25 se confundieron con la clase 2**, y de las 54 de la clase 2, **20 se asignaron a la clase 1**. SVM reduce sustancialmente ese error cruzado (11 y 15 confusiones respectivamente), aunque sigue siendo su principal fuente de fallos. Por el contrario, las clases 3 y 4 se reconocen con muy pocos errores en ambos modelos.

Interpretación geométrica de las confusiones

Esto tiene sentido por el tipo de datos: las clases 1 y 2 son **automóviles de pasajeros** con siluetas casi iguales, así que sus medidas se mezclan y cuesta separarlas. En cambio, las clases 3 y 4 (vehículos más grandes, como un autobús y una furgoneta) tienen formas muy distintas y se reconocen sin problema. Como KNN se guía solo por la cercanía entre ejemplos, esa mezcla lo afecta mucho; SVM, gracias al kernel RBF, traza fronteras más flexibles que separan mejor esos casos parecidos, aunque sin quitar del todo la confusión.

3.3. Cálculo manual de las métricas

Las métricas anteriores también pueden obtenerse **manualmente** a partir de la matriz de confusión. Para un problema de varias clases, cada clase se evalúa bajo el esquema *una contra el resto* (*one-vs-rest*), definiendo para la clase analizada cuatro cantidades: verdaderos positivos (TP), falsos negativos (FN), falsos positivos (FP) y verdaderos negativos (TN). Con ellas se aplican las fórmulas vistas en clase:

$$\text{Exactitud} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precisión} = \frac{TP}{TP + FP}, \quad (5)$$

$$\text{Sensibilidad (recall)} = \frac{TP}{TP + FN}, \quad \text{Especificidad} = \frac{TN}{TN + FP}. \quad (6)$$

3.3.1. Desarrollo para el modelo SVM

A modo de ejemplo, se desarrolla el cálculo para la **clase 1** del modelo SVM. De su matriz de confusión se obtiene $TP = 42$, $FN = 11$, $FP = 16$ y $TN = 143$:

$$\begin{aligned} \text{Precisión} &= \frac{42}{42 + 16} = 0.7241, & \text{Sensibilidad} &= \frac{42}{42 + 11} = 0.7925, \\ \text{Especificidad} &= \frac{143}{143 + 16} = 0.8994. \end{aligned}$$

Aplicando el mismo procedimiento a las cuatro clases se obtiene la Tabla 4. La exactitud global se calcula como el total de aciertos (la diagonal de la matriz) entre el total de instancias: $\frac{42+39+54+48}{212} = \frac{183}{212} = 0.8632$.

Tabla 4: Cálculo manual de métricas por clase para el modelo SVM (esquema una contra el resto).

Clase	TP	FN	FP	TN	Precisión	Sensibilidad	Especificidad
1	42	11	16	143	0.7241	0.7925	0.8994
2	39	15	11	147	0.7800	0.7222	0.9304
3	54	1	1	156	0.9818	0.9818	0.9936
4	48	2	1	161	0.9796	0.9600	0.9938

3.3.2. Desarrollo para el modelo KNN

El mismo procedimiento se aplica al modelo KNN. Para su **clase 1** la matriz de confusión arroja $TP = 23$, $FN = 30$, $FP = 23$ y $TN = 136$:

$$\begin{aligned} \text{Precisión} &= \frac{23}{23 + 23} = 0.5000, & \text{Sensibilidad} &= \frac{23}{23 + 30} = 0.4340, \\ \text{Especificidad} &= \frac{136}{136 + 23} = 0.8553. \end{aligned}$$

El detalle por clase se muestra en la Tabla 5. Su exactitud global es $\frac{23+31+53+42}{212} = \frac{149}{212} = 0.7028$. En ambos modelos los valores obtenidos a mano coinciden con los entregados por la implementación, lo que verifica la consistencia de los resultados.

Tabla 5: Cálculo manual de métricas por clase para el modelo KNN (esquema una contra el resto).

Clase	TP	FN	FP	TN	Precisión	Sensibilidad	Especificidad
1	23	30	23	136	0.5000	0.4340	0.8553
2	31	23	26	132	0.5439	0.5741	0.8354
3	53	2	8	149	0.8689	0.9636	0.9490
4	42	8	6	156	0.8750	0.8400	0.9630

Al comparar ambas tablas se ve con números lo mismo que mostraban las matrices de confusión: las clases 1 y 2 tienen la precisión y la sensibilidad más bajas porque se confunden entre sí, y justo ahí es donde SVM le gana a KNN (por ejemplo, la sensibilidad de la clase 1 sube de 0.4340 a 0.7925). Las clases 3 y 4 obtienen valores cercanos a 1 en ambos modelos. La especificidad se mantiene alta en todas las clases porque, al haber cuatro categorías parejas, la mayoría de los ejemplos pertenece a “las otras clases” y son fáciles de descartar.

4. Conclusiones

El experimento muestra que, para el conjunto *Statlog (Vehicle Silhouettes)*, **SVM con kernel RBF le gana claramente a KNN**, con una exactitud de 86.32 % frente a 70.28 % y un *Macro F1* de 0.8646 frente a 0.6985. La razón es que SVM puede trazar fronteras curvas y más flexibles, algo decisivo cuando varias clases se parecen mucho, como pasa con los dos tipos de automóvil.

KNN, aunque es simple y rápido de entrenar, tiene dos debilidades con este tipo de datos: se confunde cuando las clases se parecen demasiado y se vuelve lento al predecir conforme crecen los datos, porque debe compararse contra todos los ejemplos. SVM, en cambio, tarda más en entrenar pero predice rápido y separa las clases de forma más sólida.

En la práctica, los dos algoritmos necesitan lo mismo para funcionar bien: **estandarizar** las variables. La forma de trabajar —repartir bien los datos, elegir los parámetros con validación cruzada y evaluar con datos nuevos— permitió comparar los modelos de manera justa. En resumen, para este tipo de problemas SVM es la opción más confiable, mientras que KNN sigue siendo un buen punto de partida por lo fácil que es de entender y aplicar.